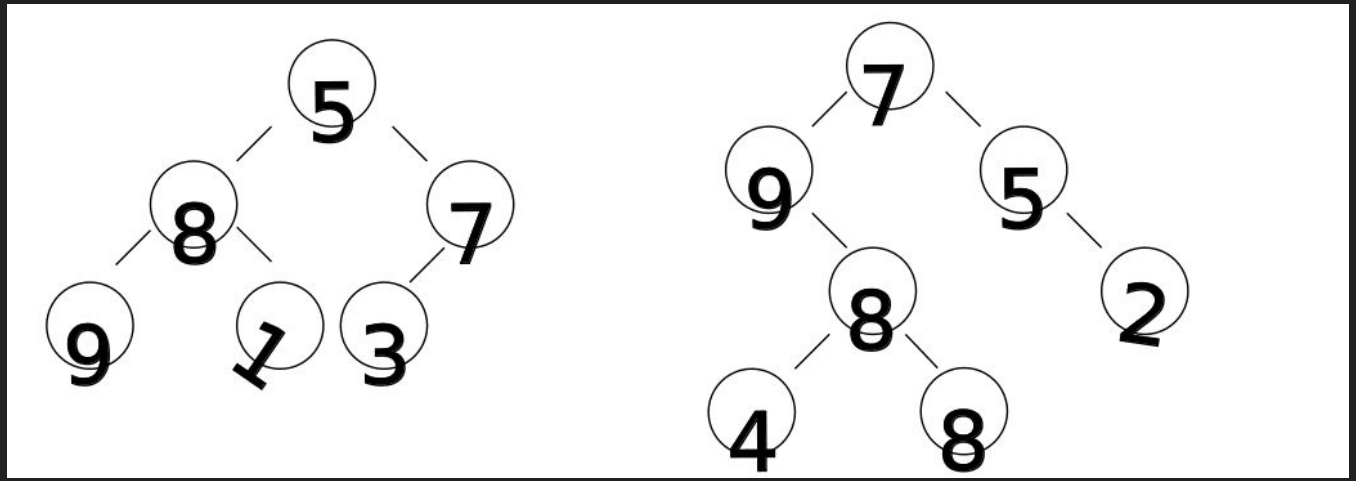
Árvores e Tabela Hash

Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza

Árvores Binárias

- Árvores Binárias (AB): um conjunto finito T de nós ou vértices, onde existem:
  - um nó especial chamado raiz
  - demais nós que podem ser divididos em dois subconjuntos disjuntos
    - sub-árvores esquerda e direita que também são AB
  - Em particular T pode ser vazio



Formalizações para ABs

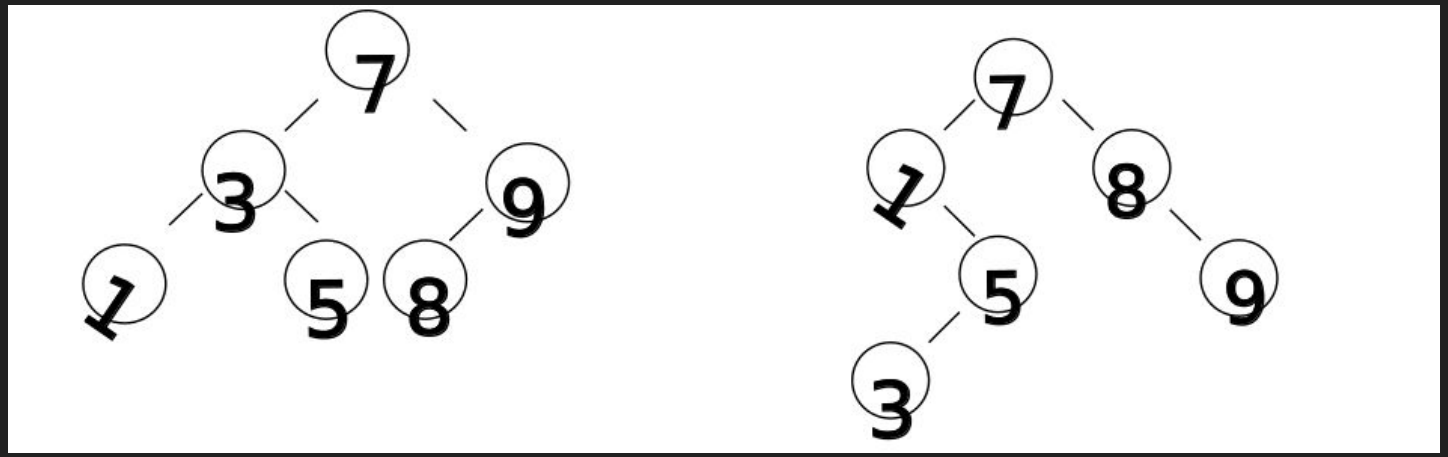
- Para cada nó de uma AB:
  - 0, 1 ou 2 filhos -> existe uma hierarquia entre os nós. Com exceção da raiz, todo nó tem um nó pai.
- Dizemos que o nível da raiz é 1 e que o nível de um nó é o nível de seu pai mais 1. A altura de uma AB é o maior dos níveis de seus nós
  - (depende da bibliografia -> normalmente, a altura da raiz é 0)
- Dizemos que um nó é folha da AB se não tem filhos.

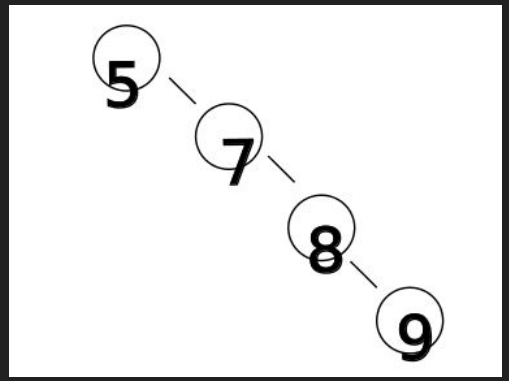
Árvores de Busca Binária (ABB)

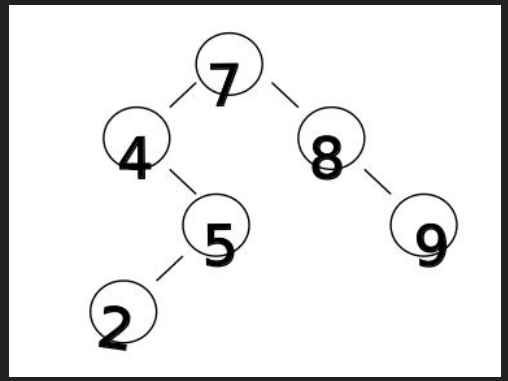
Formalização:

- Seja T uma AB. Se v é um nó de T, chamamos de info(v) a informação armazenada em v
- T é, então, uma **Árvore Binária de Busca (ABB)** se:
- v1 pertencente à **sub-árvore esquerda** de v, e **info(v1) < info(v)**.
- v2 pertencente à **sub-árvore direita** de v, e **info(v2) > info(v)**.

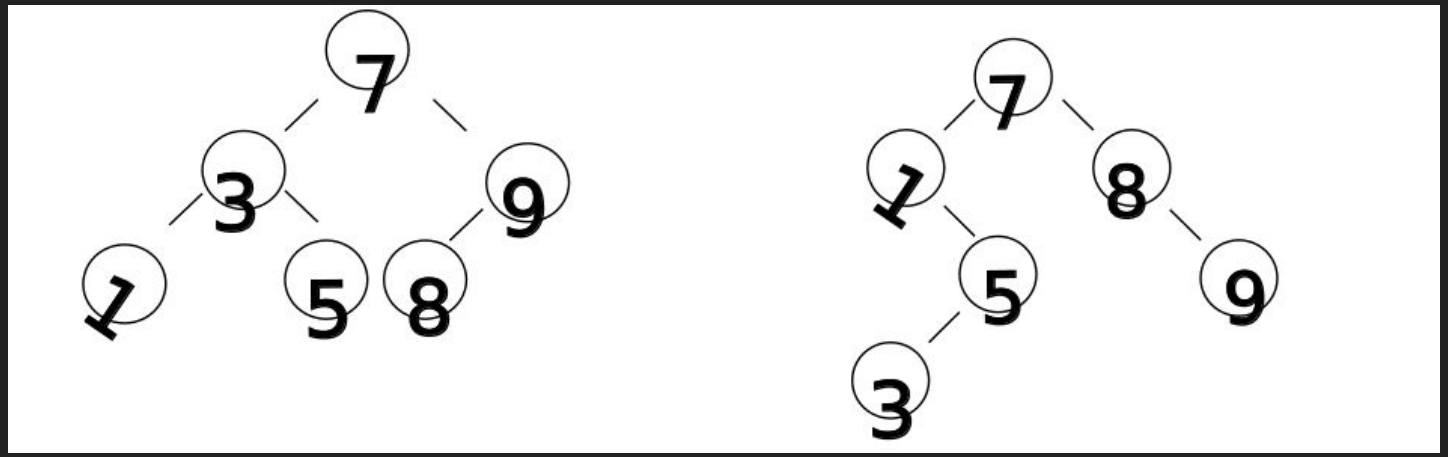
Árvores de Busca Binária

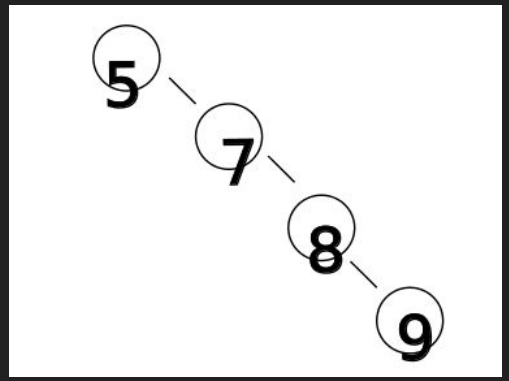


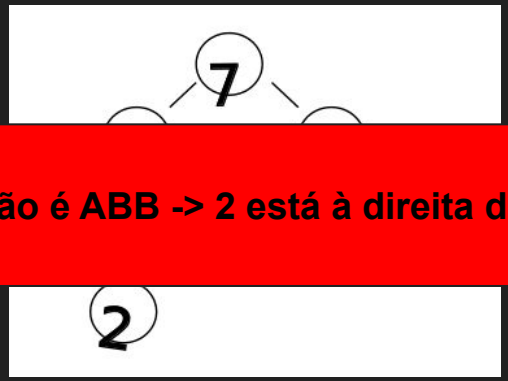




Árvores de Busca Binária







Não é ABB -> 2 está à direita do 4.

Árvores de Busca Binária

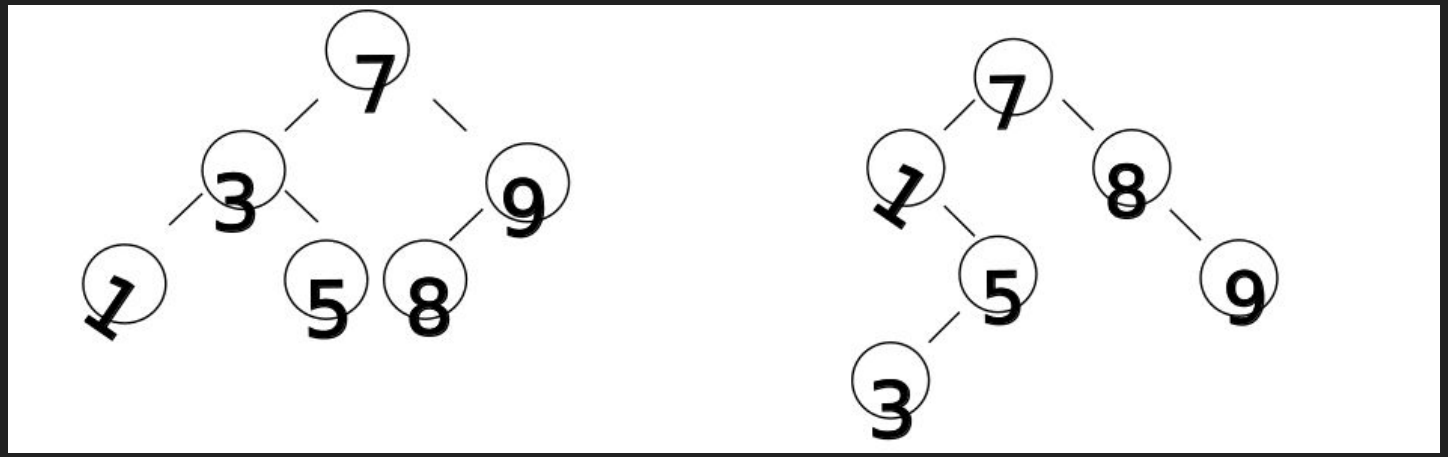
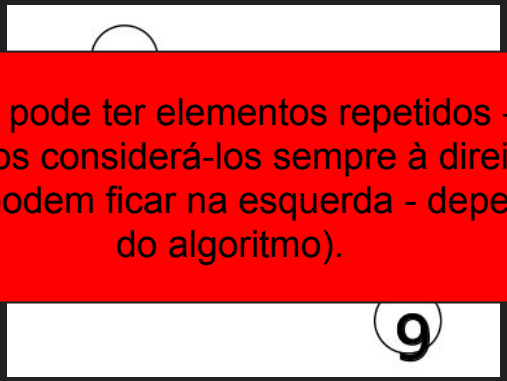
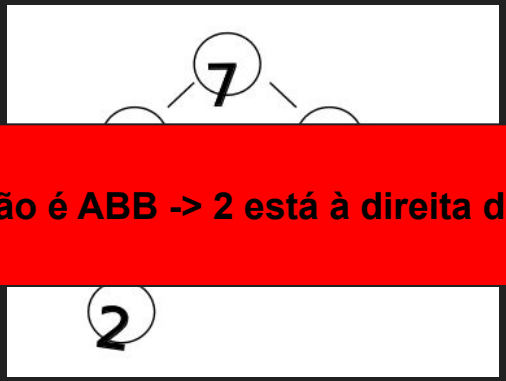


ABB pode ter elementos repetidos -> vamos considerá-los sempre à direita (mas podem ficar na esquerda - depende do algoritmo).





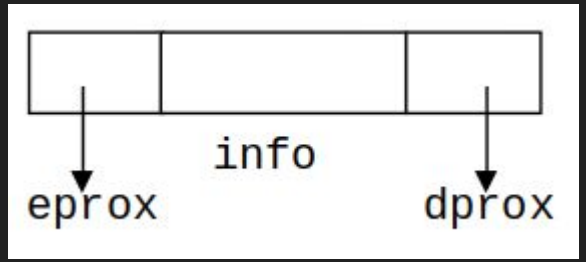
Não é ABB -> 2 está à direita do 4.

ABB com Listas Encadeadas

ABB pode ser representada por listas encadeadas:

- **info** - campo de informação
- **eprox** - ponteiro para a sub-árvore esquerda
- **dprox** - ponteiro para a sub-árvore direita

Portanto, podemos representar uma ABB como um nó que é a raiz e duas referências para as respectivas sub-árvore esquerda e direita:



Classe ABB

```
class ABB:


   def __init__ (self, raiz):
       # cria uma nova ABB com o info raiz e sem filhos
       self._info = raiz
       self._eprox = None
       self._dprox = None
```

Inserção em uma ABB

- Elementos são inseridos sempre como uma folha de uma ABB
- É necessário descer na ABB até encontrar o nó pai deste novo nó

Uma versão não recursiva para a inserção na ABB:

- A função insere(h, x) insere um elemento com info igual a x em seu devido lugar dentro da ABB h
- Se o parâmetro h for None, a ABB não existe ainda e este é o primeiro elemento
- Se x já estiver em algum nó da ABB, a inserção será à direita.

```
abb = None
for k in [4, 7, 85, 98, 4, 5, 6]: abb =
insere(abb, k)
```

```
# Insere elemento com info x na ABB h
# Elementos iguais sempre a direita
def insere(h, x):
    # verifica se será o primeiro
    if h is None:
        h = ABB(x)
        return h
    # procura o lugar para inserir x
    # percorre a ABB até achar um nó com o filho None
    p, q = h, h
    while q is not None:
        v = q._info
        p = q
        # à esquerda ou à direita
        if x < v: q = q._eprox  # esquerda
        else: q = q._dprox      # direita
    # Neste ponto, p é o pai do nó a ser inserido
    # Verificar novamente se é a esquerda ou direita
    q = ABB(x)
    if x < p._info: p._eprox = q
    else: p._dprox = q
    return h
```

Inserção em uma ABB

Exercício:

1. Implemente a versão recursiva da inserção.

Busca em ABB

- Função que procura elementos com info igual a v numa ABB h
- Versão recursiva:

```
# Procura elemento com info igual a v na ABB h
# Versão recursiva
def busca(h, v):
    if h is None: return None
    t = h._info
    if t == v: return h
    if v < t:
        return busca(h._eprox, v) # procura à esquerda
    else:
        return busca(h._dprox, v) # procura à direita
```

Busca em ABB

- Função que procura elementos com info igual a v numa ABB h
- Versão iterativa:

```
# Procura elemento com info igual a v na ABB h
# Versão iterativa
def buscaNR(h, v):
    p = h
    while p is not None:
        t = p._info
        if v == t: return p # encontrou
        if v < t: p = p._eprox   # à esquerda
        else: p = p._dprox       # à direita
    # se chegou aqui é porque não encontrou
    return None
```

Contagem de elementos em ABB

- A função conta(r, x) devolve o número de elementos iguais a x na ABB
- Versão recursiva (elementos iguais podem estar à direita ou esquerda - varre toda a ABB)

```
# conta elementos com info igual a v na ABB h
def conta(h, v):
    if h is None: return 0
    # verifica se conta este nó
    if v == h._info: a = 1
    else: a = 0
    # conta esta nó mais as ABBs esquerda e direita
    return a + conta(h._eprox, v) + conta(h._dprox, v)
```

Contagem de elementos em ABB

Exercícios

2. Faça a contagem de elementos na ABB supondo que elementos iguais estarão sempre à direita.

3. Implemente a versão iterativa da contagem de elementos na ABB.

Contagem de nós em ABB

- Contagem do número de nós de uma ABB
- Versão recursiva

```
# Conta o número de nós de uma ABB h
def contaNN(h):
    if h is None: return 0
    return 1 + contaNN(h._eprox) + contaNN(h._dprox)
```

Contagens em ABB

Exercícios

4. Implemente conta1(h) que conta o número de folhas de uma ABB
5. Implemente conta2(h) que conta o número de nós que tenham pelo menos um filho de uma ABB
6. Implemente conta3(h, x) que conta número de elementos com info >= x de uma ABB h

Impressão (em ordem) de ABB

- Função que percorre a ABB, visitando a sub-árvore esquerda, a raiz e a sub-árvore direita
- Nesta ordem, os nós serão visitados em ordem crescente do campo info (percurso em-ordem)

```
def ImprimeABB(h):
    if h is None: return
    ImprimeABB(h._eprox)
    print(h._info)
    ImprimeABB(h._dprox)
```

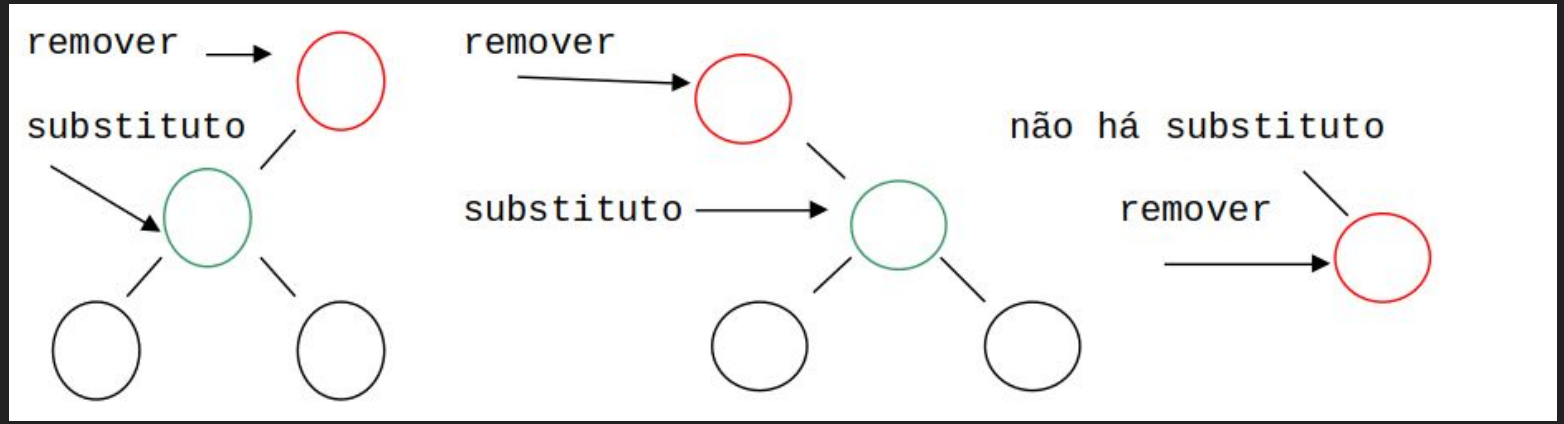
Impressão de ABB

Exercícios

7. Implemente o percurso pré-ordem em ABB (imprime raiz, visita nó esquerdo e visita nó direito)
8. Implemente o percurso pós-ordem em ABB (visita nó esquerdo, visita nó direito e finalmente visita a raiz)

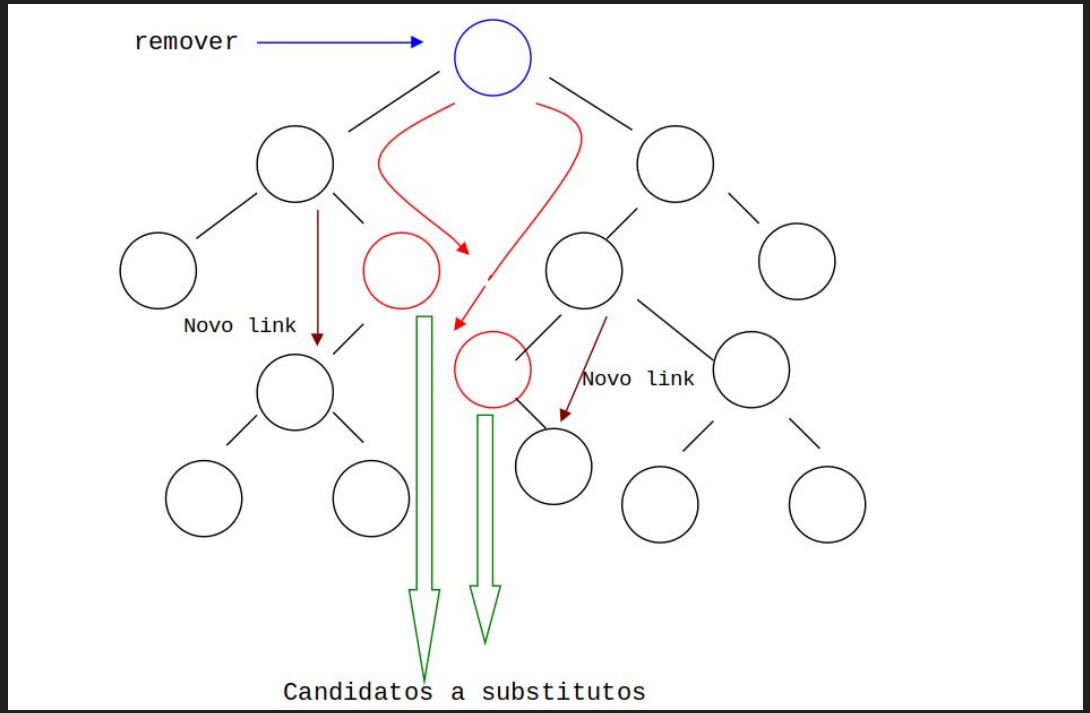
Remoção em ABB

- O problema da remoção de um nó: é necessário encontrar outro nó para substituir o removido, caso o nó a ser removido tenha filhos
- Caso 1: O nó a ser removido não tem filhos esquerdo e/ou direito



Remoção em ABB

- Caso 2: O nó a ser removido tem filhos direito e esquerdo:



Remoção em ABB

Os candidatos a substituto são obtidos percorrendo-se a ABB:

- Um a esquerda e tudo a direita até achar nó com **dprox NULL**
- Um a direita e tudo a esquerda até achar nó com **eprox NULL**
- Determinado o candidato a substituir, é preciso então:
  - Substituir o conteúdo (campo info) do nó a ser substituído pelo do candidato
  - Como o candidato pode ter filhos (apenas esquerdo ou apenas direito), esses filhos serão herdados pelo pai do candidato -> basta alterar o ponteiro desse nó pai do candidato para o seu filho

Altura de ABB

- A altura de uma ABB deve ser a maior altura de seus nós folha
- Implementação recursiva:

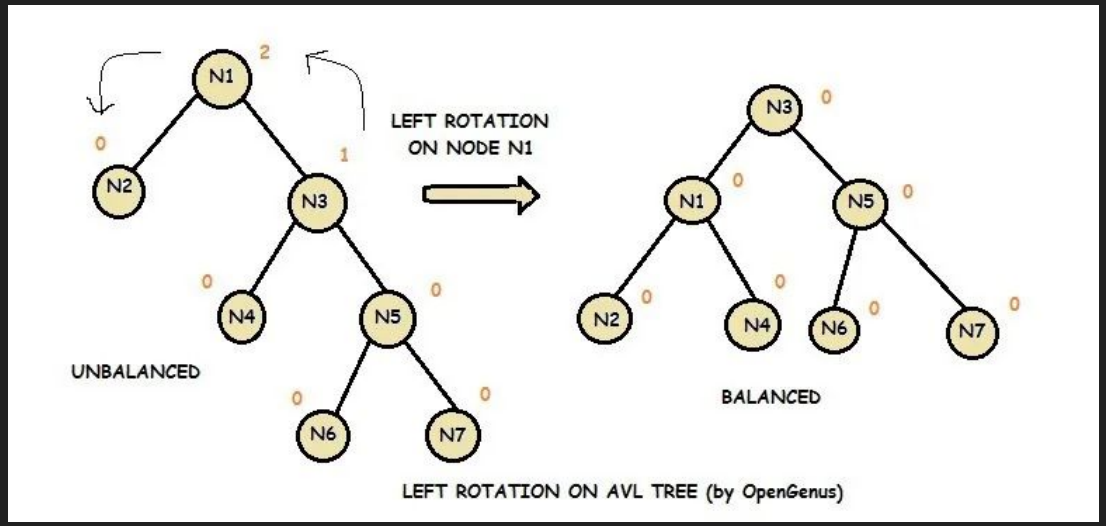
```
# altura de uma ABB
def altura(h):
   if h is None: return 0

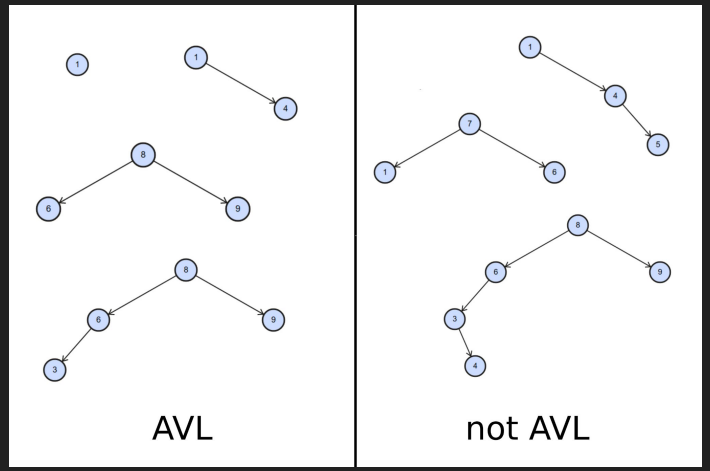
   alturaEsquerda = altura(h._eprox)
   alturaDireita = altura(h._dprox)

   return max(alturaEsquerda, alturaDireita) + 1
```

Outras Árvores

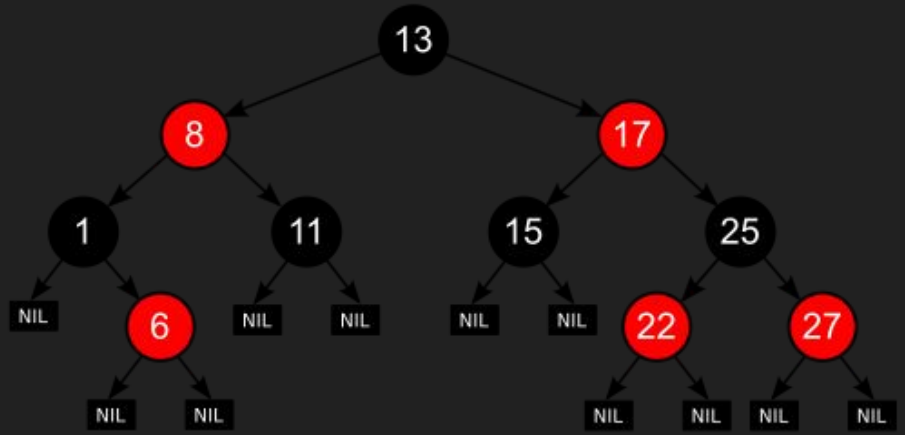
- Árvores Binárias de Busca AVL (Adelson-Vesky e Landis (1962))
  - Cada nó mantém uma informação adicional, chamada fator de balanceamento que indica a diferença de altura entre as sub-árvores esquerda e direita.
  - As operações de inserção e remoção mantém o fator de balanceamento entre -1 e +1.





Outras Árvores

- Árvores Binárias de Busca Rubro-Negras
  - Todo nó é vermelho ou preto
  - Todo null é preto
  - Se um nó é vermelho então seus filhos são pretos
  - Todo caminho da raiz até qualquer folha tem sempre o mesmo número de nós pretos.
- Mantêm a ABB mais ou menos balanceada após inserções e remoções



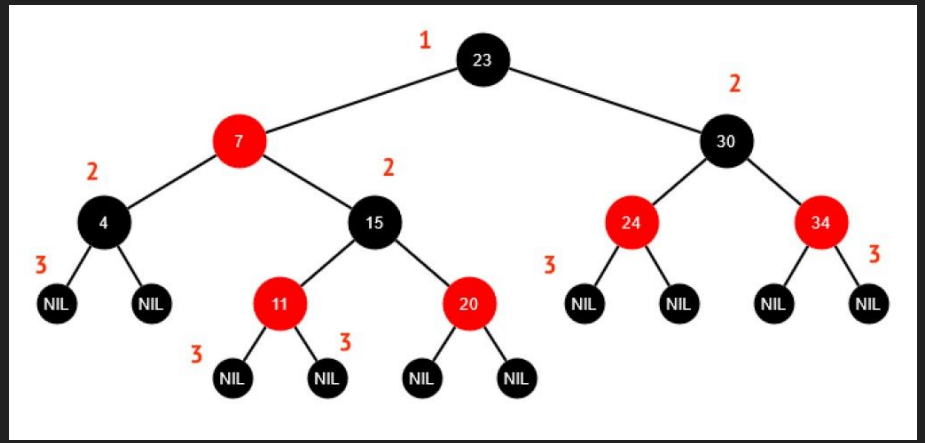


Tabela Hash

- Uma forma de busca em uma tabela: dividi-la em várias sub-tabelas de tal forma que seja possível determinar a priori em qual sub-tabela estará cada elemento -> a busca se restringe a uma quantidade menor de elementos.
- Uma simplificação desta técnica é determinar a posição do elemento dentro da tabela através de seu valor. Definimos então uma função que transforma o valor do elemento numa posição da tabela (função de hash) -> basta verificar se o elemento está ou não nesta posição
- O objetivo então é transformar a chave de busca em um índice na tabela

Exemplo: Construir uma tabela com os elementos 34, 45, 67, 78, 89. Vamos usar uma tabela com 10 elementos e a função de hash (x % 10) (resto da divisão por 10). A tabela ficaria.

i	0	1	2	3	4	5	6	7	8	9
a[i]	None	None	None	None	34	45	None	67	78	89

Implementação basal de Tabela Hash

```
def hash(x):
 return x % 10

def insere(a, x):
  a[hash(x)] = x

def busca_hash(a, x):
  k = hash(x)
  if a[k] == x: return k
  return -1
```

Tabela Hash

- Exemplo: elementos 23, 42, 33, 52, 12, 58
- Com a mesma função de hash, teremos mais de um elemento para determinadas posições (colisões - 42, 52 e 12; 23 e 33)
- Pode-se alterar a função de hash para (x % 17) -> uma tabela de 17 posições

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a[i]	None	52	None	None	None	None	23	58	42	None	None	None	12	None	None	None	33

- A escolha da função é a parte mais importante -> deve-se priorizar o uso do espaço útil da tabela
- É um compromisso (trade-off) entre a eficiência na busca o gasto de memória.

Colisões em Tabela Hash

- No caso geral: não tem-se informações sobre os elementos e seus valores
  - Problema: como tratar os elementos cujo valor da função de hash é o mesmo? -> Colisões
- Existe uma forma simples de tratar o problema das colisões: colocar o elemento na primeira posição livre seguinte e considerar a tabela circular (o elemento seguinte ao último a[n-1] é o primeiro a[0])
  - Conceito propagado da inserção de novos elementos para a busca
- Considerando os elementos anteriores e a função (x % 10). A tabela ficaria: 23, 42, 33, 52, 12, 58

I	0	1	2	3	4	5	6	7	8	9
a[i]	None	None	42	23	33	52	12	None	58	None

Implementação estendida de Tabela Hash

```
def hash(x, M):
  return x % M
def insere_hash(a, x):
   M = len(a)
   cont = 0
   i = hash(x, M)
   # procura a próxima posição livre
   while a[i] != None:
       if a[i] == x: return -1 # valor já existente na tabela
       cont += 1               # conta os elementos da tabela
       if cont == M: return -2 # tabela cheia
       i = (i + 1) % M         # tabela circular
   # achamos uma posição livre - coloque x nesta posição
   a[i] = x
   return i
def busca_hash(a, x):
   M = len(a)
   cont = 0
   i = hash(x, M)
   # procura x a partir da posição i
   while a[i] != x:
       if a[i] == None: return -1  # não achou x, pois há uma vazia
       cont += 1                   # conta os elementos da tabela
       if cont == M: return -2;    # a tabela está cheia
       i = (i + 1) % M             # tabela circular
```

Função Hash

- Compromisso entre o tamanho máximo e a quantidade estimada de comparações necessárias para se encontrar o elemento
- Operação módulo (%): a maneira mais direta de transformar valores em índices:
  - Se tivermos um conjunto de inteiros e uma tabela de M elementos, a função de hash pode ser simplesmente (x % M).
  - Se tivermos um conjunto de valores fracionários entre 0 e 1 com 8 dígitos significativos, a função de hash pode ser (x * 10^8 % M).
  - Se houver mais que 8 dígitos, podem considerar só os 8 primeiros (int(x * 108) % M)
  - Se forem números entre s e t, a função pode ser ((x - s) * (M – 1)) / (t - s) arredondando ou truncando para o inteiro mais próximo.
- Escolha de função livre, porém, objetivo é sempre espalhar elementos ao máximo dentro da tabela -> minimizar as colisões -> expectativa de comportamento uniformemente distribuído das chaves

Tratamento de Colisões

1. Lista linear simples - Busca sequencial

- Como anteriormente, todos os elementos compartilham a mesma lista e se ao inserir um elemento a posição estiver ocupada, procura-se a próxima posição livre à frente -> lista circular
- No pior caso, no qual a lista está completa (M elementos ocupados), teremos que percorrer os M elementos
  - O algoritmo é **O(M) -> diretamente relacionado ao tamanho da tabela**
  - Entretanto, quando se tem alguma informação sobre os elementos, é possível construir uma função de hash sobre uma tabela que busque minimizar o número de comparações para se encontrar um inserir um novo elemento
  - É uma situação bem melhor que a simples busca sequencial. Note que melhor que isso seria fazer busca binária, mas para isso os dados teriam que chegar em ordem. Não é o caso geral. Os dados podem ser inseridos à medida que chegam, numa tabela de hash.

Tratamento de Colisões

2. Duplo Hash

- ⬆ espalhamento: deslocamento dado por uma segunda função de hash
  - Segunda função de hash tem que ser escolhida com cuidado
  - Não pode dar zero (loop infinito -> colisões)
  - Deve ser tal que a soma do índice atual com o deslocamento (módulo M) dê sempre um número diferente até que os M números sejam verificados. Para isso **M** e o **valor desta função** devem ser **primos entre si** -> escolher M primo e garantir que a segunda função de hash tenha um valor constante K menor que M e maior que 1

Tratamento de Colisões

```
def hash(x, M):
  return ...  # valor da função

def hash2():
  return ...  # valor da função - passo

def insere_hash(a, x):
   M = len(a)
   cont = 0
   i = hash(x, M)
   k = hash2()
   # procura a próxima posição livre
   while a[i] != None:
       if a[i] == x: return -1 # valor já existente na tabela
       cont += 1               # conta os elementos da tabela
       if cont == M: return -2 # tabela cheia
       i = (i + k) % M         # tabela circular
   # achamos uma posição livre - coloque x nesta posição
   a[i] = x
   return i
```

Tratamento de Colisões

```
def busca_hash(a, x):
    M = len(a)
    cont = 0
    i = hash(x, M)
    k = hash2()
    # procura x a partir da posição i
    while a[i] != x:
        if a[i] == None: return -1  # não achou x, pois há uma vazia
        cont += 1                   # conta os elementos da tabela
        if cont == M: return -2;    # a tabela está cheia
        i = (i + k) % M             # tabela circular
    # encontrou
    return i
```

- O duplo hash espalha mais os elementos, mas no pior caso continua sendo **O(M)**

Tabela Hash com sublistas

- Pode-se construir uma tabela hash contendo sub-listas com os elementos que têm o mesmo valor da função de hash
- Usando como exemplo uma lista com 23, 42, 33, 52, 12, 58 (inseridos nesta ordem) em uma tabela de 10 elementos e (x % 10) como função de hash, a lista contendo a tabela ficaria:

```
[None, None, [42, 52, 12], [23, 33], None, None, None, None, 58, None]
```

Tabela Hash com sublistas

```
def hash(x, M):
  return x % M
# Insere x na tabela de hash a a[i] é um elemento da tabela ou uma outra lista contendo os #
# elementos com a mesmo valor de hash. Devolve (None, None) - não inseriu porque já estava
#          (i, None) - se inseriu em a[i]
#          (i, j) - se inseriu em a[i][j]
def insere_hash(a, x):
  M = len(a)
  i = hash(x, M)
  # tentar inserir x na tabela
  if a[i] is None:
     a[i] = x
     return (i, None)
  # se a[i] é uma lista
  if type(a[i]) is list:
     # procura x em a[i]
     if x in a[i]:
         return (None, None) # x já está na tabela
     # pode inserir x na lista a[i]
     k = len(a[i])
     a[i].append(x)
     return (i, k)
  # a[i] é um elemento simples
  if a[i] == x:
     return (None, None) # já está
  # iniciar a lista em a[i] e inserir elemento
  a[i] = [a[i], x]
  return(i, 1)
```

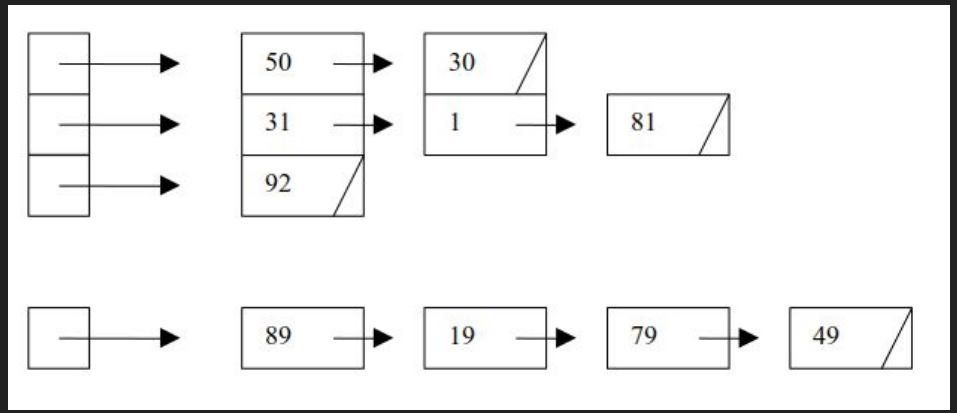
Tabela Hash com sublistas

```
# Procura x na tabela de hash a. Devolve (None, None) - se x não está na
tabela
#          (i, None) - se x == a[i]
#          (i, j) - se x == a[i][j]
def busca_hash(a, x):
  M = len(a)
  i = hash(x, M)
  # x não está na tabela
  if a[i] is None:
     return (None, None)
  # se a[i] é uma lista
  if type(a[i]) is list:
     # procura x em a[i]
     for k in range(len(a[i])):
        if x == a[i][k]:
           return (i, k)
     # não encontrou
     return (None, None)
  # a[i] é um elemento simples
  if a[i] == x:
     return (i, None) # encontrou
  # não encontrou
  return(None, None)
```

Tabela Hash com Listas Encadeadas

- Pode-se criar uma LE com os elementos que têm o mesmo valor da função de hash, em vez de deixá-los todos na mesma tabela
- O efeito é o mesmo da solução anterior: diminuir o número de comparações para se procurar um elemento
- Tem-se uma lista sentinela com as referências iniciais de cada LE

Como exemplo, considere uma tabela de inteiros e como função de hash (x % 10).



Implementação

```
# classe _Node
class Node:
  def __init__ (self, info, prox):
     # inicia os campos
     self._info = info
     self._prox = prox
class TabelaHashListaLigada:
   # métodos da classe
   def __init__ (self, M):
       # cria M LEs vazias
       self._numheads = M
       self._head = [None] * M
   def hash(self, x, M):
       return x % M
   def insere(self, e):
       # adiciona elemento no inicio da LE
       # adiciona sempre - inclusive se já estiver
       # novo nó referencia o inicio da LE
       k = self.hash(e, self._numheads)
       novo = Node(e, self._head[k])
       # novo nó será o inicio da LL
       self._head[k] = novo
```

Implementação

```
# classe _Node
class Node:
  def __init__ (self, info, prox):
     # inicia os campos
     self._info = info
     self._prox = prox
class TabelaHashListaLigada:
   # métodos da classe
   def __init__ (self, M):
       # cria M LEs vazias
       self._numheads = M
       self._head = [None] * M
   def hash(self, x, M):
       return x % M
   def insere(self, e):
       # adiciona elemento no inicio da LE
       # adiciona sempre - inclusive se já estiver
       # novo nó referencia o inicio da LE
       k = self.hash(e, self._numheads)
       novo = Node(e, self._head[k])
       # novo nó será o inicio da LL
       self._head[k] = novo
```

Pode ser melhorada: Só inserir se já não estiver na tabela. Para isso seria necessário percorrer a lista até o final; Inserir elemento de modo que a lista fique em ordem crescente.

Implementação

```
def procura(self, e):
    # procura elemento com info = e.
    # devolve referência para esse elemento ou None se não acha
    k = self.hash(e, self._numheads)
    # p percorre a lista
    p = self._head[k]
    while p is not None:
        if p._info == e: return p # achou
        p = p._prox # vai para o próximo
    # se chegou aqui é porque não achou
    return None

def MostraTabHash(self):
    # mostra cada uma das LEs
    for k in range(self._numheads):
        print("\nLista ", k)
        p = self._head[k]
        while p is not None:
            print("***", p._info)
            p = p._prox
```

Implementação

```
#Testes
# cria tabela de valores
tabela = [34, 54, 89, 98, 134, 85,99]
print("tabela de elementos: \n", tabela)
# Cria tabela hash ligada com NumLL LLs e insere valores
NumLL = 5
TabHash = TabelaHashListaLigada( NumLL)
# insere os elementos de tabela
for k in tabela:
   TabHash.insere(k)
# mostra a tabela construida
TabHash.MostraTabHash()
# teste - procura elementos
print("\nteste - procura elementos" )
if TabHash.procura(33) is None:
  print("não achou")
if TabHash.procura(55) is None:
  print("não achou")
for elem in tabela:
  if TabHash.procura(elem) is None:
     print(elem, " * não achou")
  else:
     print(elem, " * achou")
```

Tabelas Dinâmicas

- Busca sempre mais demorada à medida que a tabela fica mais populada
  - Enquanto a tabela está com poucos elementos, a busca tende a ser muito rápida
- **Solução:** aumentar o seu tamanho quando começar a ficar cheia
  - Aumentar a tabela nunca é uma operação eficiente -> todos os elementos devem ser inseridos novamente
  - Pode valer a pena se operação é realizada poucas vezes
- Adotando uma convenção para uma tabela de N elementos: quando a tabela passa de N/2 elementos, o tamanho é dobrado
  - A tabela sempre terá menos da metade de seus elementos ocupados

Tabelas Dinâmicas

```
# Inicia as variáveis globais da tabela hash dinâmica
M = 100      # tamanho da tabela
MM = 0       # novo tamanho da tabela
N = 0        # quantidade de elementos da tabela
p = [None] * M    # esta é a tabela de hash em uso
q = []       # nova tabela de hash

def hash(x, T):
  # devolve a função de hash para o elemento x e tabela de tamanho T
  return x % T

# insere novamente x na nova tabela q
def insere_nova(x):
  global M, MM, N, p, q
  i = hash(x, MM)
  # procura a próxima posição livre
  # sempre tem lugar, pois a tabela foi duplicada
  while q[i] != None:
     i = (i + 1) % MM # tabela circular
  # achamos uma posição livre
  q[i] = x
  return i
```

Tabelas Dinâmicas

```
# expande a tabela para o dobro do tamanho
def expande():
  global M, MM, N, p, q
  MM = 2 * M
  q = [None] * MM  # nova lista com o dobro de elementos
  # insere todos os elementos na tabela nova
  for i in range(M):
     if p[i] != None: insere_nova(p[i])
  # novos valores para M e p
  M = MM
  p = q  # p e q são a mesma lista
# insere elemento na tabela hash
def insere(x):
  global M, MM, N, p, q
  # verifica se o tamanho da tabela está adequado
  if M < N + N:
     # se já tem a metade cheia, é melhor expandir
     expande()
  i = hash(x, M);
  # procura a próxima posição livre - sempre vai encontrar
  while p[i] != None:
     # encontrou - insere x se já não está
     if p[i] == x: return -1    # valor já existente na tabela
     i = (i + 1) % M            # tabela circular
  # achamos uma posição livre
  p[i] = x
  N += 1
  return i;
```

Tabelas Dinâmicas

```
def busca_hash(x):
  global M, MM, N, p, q
  i = hash(x, M)
  cont = 0
  # procura x a partir da posição i
  while p[i] != x:
      if p[i] == None: return -1  # não achou x, pois há uma vazia
      cont += 1                   # conta os elementos da tabela
      if cont == N: return -2;    # esse é o último - não achou
      i = (i + 1) % M             # tabela circular
  # encontrou
  return i
```

Observações sobre Tabela Hash

- Cada um dos métodos apresentados para tratar colisões tem vantagens e desvantagens:
  - **Lista de hash simples:** é o mais rápido se o tamanho de memória permite que a tabela seja bem esparsa
  - **Duplo hash:** usa melhor a memória, mas depende de memória que permita que a tabela continue esparsa
  - **Lista encadeada ou lista com sub-lista:** é interessante, mas depende de um alocador rápido de memória
- A escolha depende da análise da aplicação
- Vantagem do Hash: se as chaves são razoavelmente conhecidas e possui-se memória suficiente -> existe função hash que possibilite um tempo aproximadamente constante para o algoritmo
  - Portanto O(1) (casos ruins do uso de Hash podem apresentar desempenho O(N), por exemplo)



Obrigado!

Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr